MUHAMMAD TARIQ

Backend Engineer (Go) · AI/LLM Application Engineering · RAG & AI Systems

+92 303 0106788 · tariqwc99@gmail.com · Islamabad, Pakistan · linkedin.com/in/mtariq99 · github.com/MTariq99

PROFESSIONAL SUMMARY

Backend engineer with 5+ years of professional experience building Go and Node.js distributed systems: gRPC microservices, event-driven pipelines (Kafka, RabbitMQ) and real-time platforms. Transitioning into LLM application engineering through two personal Go projects (working prototypes): a multi-tenant RAG backend with hybrid pgvector and full-text retrieval, citations and an evaluation pipeline, and a signed-request LLM orchestration engine with idempotent tool calling. Focused on integrating LLMs into reliable, secure and observable backend systems.

TECHNICAL SKILLS

Languages: Go, TypeScript, JavaScript (Node.js), SQL

Backend & Distributed Systems: REST APIs, gRPC, microservices, event-driven architecture, WebSockets, Socket.io, Clean Architecture, system design

Databases & Messaging: PostgreSQL, Redis, MySQL, MongoDB, Apache Kafka, RabbitMQ

AI / LLM Engineering (personal projects): RAG, hybrid retrieval (pgvector + PostgreSQL full-text search), Reciprocal Rank Fusion, embeddings, LLM tool calling and orchestration, grounded generation with citations, SSE streaming, retrieval evaluation (Precision@K, Recall@K, MRR), LLM-as-judge evaluation, Gemini API

Security: Request signing and replay protection (ECDSA P-256), opaque-token and JWT authentication, permission-based access control (personal projects)

Observability & Reliability: Prometheus, Grafana, blue-green deployments; OpenTelemetry, Sentry (personal projects)

Infrastructure & Tools: Docker, Docker Compose, CI/CD, Git, Linux, Temporal, Gin, Express.js

PROFESSIONAL EXPERIENCE

Target Technologies Pakistan
July 2025 – Present

Backend Engineer (Go)
Islamabad · On-site

- Develop and maintain Go backend services across multiple Government of Oman projects, including WebSocket-driven live driver and cargo tracking with location updates refreshed every five seconds.
- Work within an architecture of 10+ gRPC microservices (SSO, configurator, profiler, email sender, notification sender), updating service contracts as product requirements change.
- Build APIs around existing route-optimization and delivery-scheduling services.
- Integrate payment gateway functionality through gRPC clients connecting to existing payment services.

ZAMH Technologies LLC
September 2024 – June 2025

Backend Developer (Go & Node.js)
New York, NY · Remote

- Developed and maintained backend services using Go and Node.js, implementing APIs and business logic to support application requirements.
- Optimized database queries and schema design, applying indexing and connection-pooling improvements to support more efficient data access.
- Implemented Redis caching to reduce repeated database work and improve backend responsiveness.
- Built real-time chat and notification functionality using Node.js and WebSockets.
- Monitored application performance and service health using Prometheus and Grafana.
- Investigated production issues, performed root-cause analysis, and improved code reliability through refactoring, code reviews, and testing.

CapregSoft Ltd.
January 2023 – August 2024

Backend Developer (Go & Node.js)
Wah Cantt · On-site

- Developed backend services and APIs using Go and Node.js to support application features and business workflows.

- Built backend functionality for a bulk email verification platform, supporting email verification workflows and related processing.
- Improved database query performance through query restructuring, indexing, and server-side performance tuning.
- Worked on database access patterns and backend logic to improve the efficiency and maintainability of existing services.
- Operated blue-green deployment workflows to support application releases and reduce deployment risk.
- Troubleshot backend defects and performance issues, contributing to service stability and ongoing application maintenance.

Freelance Backend Engineer
April 2021 – January 2023

Backend Developer (Go) *Remote*

- Delivered scalable backend systems for clients across multiple industries in Go, designing event-driven microservice architectures with Kafka and RabbitMQ.
- Reduced data processing latency through asynchronous pipeline design and performance tuning.
- Implemented gRPC for inter-service communication across client projects.

PERSONAL AI PROJECTS

Retrievion: Multi-Tenant RAG Backend (Go)
github.com/MTariq99/retrievion

Personal Project · Working prototype · Grounded Q&A over private PDFs

- Built an asynchronous ingestion pipeline: tenant-scoped SHA-256 deduplication, versioned documents, Backblaze B2 storage and RabbitMQ jobs with a dead-letter queue; workers do page-aware parent/child chunking and batched Gemini embeddings, persisting chunks transactionally.
- Implemented hybrid retrieval in PostgreSQL: pgvector cosine search (HNSW) and full-text search (GIN) merged with Reciprocal Rank Fusion under a token-bounded context budget, keeping documents, embeddings, conversations and citations in one transactional store.
- Built a grounded chat API with SSE streaming; the prompt builder treats retrieved text as untrusted and a citation resolver maps [n] references to chunk, document and page, persisted with each answer.
- Built a dataset-driven RAG evaluation pipeline in Go that measures Precision@K, Recall@K and MRR, evaluates answer faithfulness using an LLM judge, persists per-case results and aggregate scores in PostgreSQL, and supports baseline comparisons and Prometheus evaluation metrics.
- Instrumented with Prometheus (tokens, estimated cost, latency, time-to-first-token, retrieval latency), OpenTelemetry and Sentry; transient LLM errors are retried with backoff and jitter, but streams are not retried once tokens are sent.
- *Scope: working prototype, not production-hardened; evaluation results are not reported. The reranker is a no-op; cross-encoder reranking, ingestion retries, fine-grained document permissions and CI-gated evaluation are planned.*

DispatchAI: LLM Orchestration Engine (Go)
github.com/MTariq99/dispatchai

Personal Project · Early-stage prototype · Signed host integration

- Built the orchestration core: a bounded tool-calling loop, conversation-level coordination and a versioned per-integration tool registry; tool calls are untrusted proposals executed through a host-owned gateway.
- Implemented ECDSA P-256 / SHA-256 signed integration requests with a ±5-minute timestamp window, per-integration nonce replay protection, request-body hash binding and key enrollment.
- Implemented request idempotency bound to request fingerprints, run lifecycle management, tool-call idempotency keyed by run and call ID, and policy enforcement (one rule installed); exactly-once effects across host systems are not guaranteed.
- Used ports-and-adapters design with a provider-agnostic LLM interface (Gemini adapter wired) and a reference host application demonstrating the integration contract.
- *Scope: early-stage prototype. Approval workflows, audit, observability, outbound gateway authentication, tool-argument schema validation and key rotation are planned; the host remains responsible for validating and authorizing tool calls.*

EDUCATION

International Islamic University Islamabad
Bachelor of Science, Software Engineering

LANGUAGES

English (Fluent) · **Urdu** (Fluent) · **Pashto** (Native)